FIT3077 Software Engineering: Architecture and Design

Team name: Santorini Craftsmen 001

Team leader: Emil Chin Jia Zhen (33520542)

Team members: 1. Tiffany Lau Kho Jen (33521859)

2. Pee Yee Peen (33591032)

3. Wong Xin Thung (33592357)

1.0 Team Information and Technology Prototypes

This section shows the team information, including the team name, team photo, team membership, and team schedule. Technology stack and justification are also included in this section.

1.1 Team Name and Team Photo

Team Name: Santorini Craftsmen 001
Moodle Team Name: MA_Tuesday06pm_Team001

Team Photo:

Figure 1.1.1 Santorini Craftsmen's Team Photo

The team members of Santorini Craftsmen are arranged as follows:
Leftmost: Wong Xin Thung; Middle-left: Pee Yee Peen; Middle-right: Tiffany Lau Kho Jen; Rightmost: Emil Chin Jia Zhen

1.2 Team Membership

This section provides an overview of each team member's basic personal information, such as name and contact details. It also includes the selection of a team leader, along with a list of each team member's technical and professional strengths, as well as a fun fact description.

1.2.1 Basic Personal Information

Table 1.2.1 shows the personal details of each team member, including name, student ID, student email, and contact number.

Table 1.2.1 Basic information about each team member

Team Members Name	Contact details		
	Student ID	Student Email	Contact Number
Tiffany Lau Kho Jen	33521859	tlau0015@student.monash.edu	+60107704147
Pee Yee Peen	33591032	yee0001@student.monash.edu	+601112733288
Wong Xin Thung	33592357	xwon0021@student.monash.edu	+601155190067
Emil Chin Jia Zhen	33520542	echi0039@student.monash.edu	+60143988417

1.2.2 Team Leader Selection

The team leader chosen for Sprint 1 is **Emil Chin Jia Zhen (33520542)**.

1.2.3 Technical and Professional Strengths

This section introduces the technical and professional aspects of each team member.

Member 1: Tiffany Lau Kho Jen

I am proficient in coding several programming languages such as Python, Java, HTML, CSS, and JavaScript. My expertise in Python and Java allows me to build interactive games which include implementing the game logic and applying the object-oriented programming concepts. I also utilised the frameworks and libraries available to create these games like Pygame to render graphics such as sprites, sprites animations, and the background. Moreover, I am skillful in developing responsive web-based applications, using a combination of HTML, CSS, and JavaScript. I used HTML and CSS to create user-friendly interfaces that are visually appealing and easy to navigate through. I leverage Javascript mostly to handle user interactions with event listeners as well as real-time content updates. I enjoy tackling difficult problems with an analytical mindset, allowing me to expand my knowledge and technical expertise in this field.

Member 2: Pee Yee Peen

I have extensive experience writing code across multiple languages, including Python, Java, MATLAB, SQL, and HTML. I am proficient in developing efficient applications. For instance, I utilise Python for scripting and Java for software development. Besides, I have a strong foundation in solving numerical problems and ensuring effective database management by using MATLAB and SQL respectively. I am also skilled in structuring web content using HTML. This diverse skill set allows me to adapt to different programming challenges and efficiently develop solutions accordingly.

Member 3: Wong Xin Thung

I have previously studied Python, Java, and MATLAB, among other programming languages, and have used them to finish some of my assignments. For example, I previously developed software using Python, and for my last semester assignment, I utilised Java to create a text-based game. At the same time, I am familiar with GitLab, which may enhance team collaboration efficiency by efficiently sharing and managing code with peers. Along with knowing the Agile methodology, I believe this approach can put it into practice while doing my assignments, which helped me better grasp how to move the project along and adjust to the pace of cooperation.

Member 4: Emil Chin Jia Zhen

I have a basic understanding of programming, mainly in Python and Java. Besides, I used Java to create a simple game and Python to code a game simulation, along with other assignments in previous semesters. I also know some backend coding, like working with databases and JavaScript, though I am still improving in these areas. Additionally, I have used MATLAB to solve physics and math problems and MARIE to create a simple countdown display. My strengths include problem-solving, logical thinking, and adaptability, and I am keen to keep learning and improving my skills in software and game development.

1.2.4 Fun Fact

This section highlights a unique and lesser-known fun fact about each team member.

Member 1: Tiffany Lau Kho Jen

A fun fact about me is that I like to listen to music composed of rap, hip-hop and R&B genres. Listening to them lifts my mood up instantly and helps me to stay relaxed. Besides that, it improves my focus when studying or exercising as I become more energetic and active. I also attended one of my favourite rapper’s concerts two years ago in Kuala Lumpur so that I could see him perform live on stage and gain an unforgettable experience.

Member 2: Pee Yee Peen

A fun fact about me is that I am a spicy enthusiast. I enjoy exploring different cuisines that bring the heat, as I find that spicy food not only excites my taste buds but also helps me relax when I am feeling stressed. I enjoy every moment of the spice challenge.

Member 3: Wong Xin Thung

One interesting thing about me is that I enjoy watching action, suspense, and Disney films. I find that watching films helps me relax and reduce my stress, especially comedies that always make me laugh out loud and help me forget about my problems. Every movie is like a fantastic adventure that lets me lose myself in it and enjoy each moment, whether it has a thrilling story or a heartwarming and inspiring story.

Member 4: Emil Chin Jia Zhen

A fun fact about me is that I always do better on the second try. For example, when I try to throw rubbish into a bin from a distance, I almost always miss it on the first attempt but get it in on the second. This is a fact—it has been proven time and time again that I tend to perform better on my second try. So, bear with me! XD

1.3 Team Schedule

Our team has set up a regular meeting schedule to keep communication flowing and ensure we stay on the same page. We'll be meeting **twice a week**, on **Tuesdays** and **Wednesdays** at **10:00 AM**, for about **an hour**. Depending on everyone's availability, these meetings will be held either in **person or online**, so all team members can join no matter where they are. If we are approaching a deadline or need to discuss something urgent, we will schedule **extra meetings** if needed. Each session will have a clear agenda covering progress updates, task assignments, and any roadblocks we're facing. We'll also take notes and send it to our WhatsApp group, and make meetings minutes after every meeting to keep track of action items and make sure nothing slips through the cracks.

When it comes to our regular work schedule, everyone is expected to contribute at least **10 hours per week** to the project. We will distribute tasks based on each person's strengths, availability, and expertise to make sure the workload is balanced. To stay organised, we will be using **Trello** to track tasks, deadlines, and overall progress, **Gitlab** to commit tasks done by each member. Assignments will be discussed and updated during our meetings, and everyone will be responsible for keeping their status current. If anyone runs into challenges, they're encouraged to ask for help so we can tackle issues together and keep things moving forward. By staying in sync and maintaining a structured approach, we aim to complete our work efficiently and on time.

1.4 Technology Stack and Justification

1.4.1 Selection of Programming Language

We made this decision as a team after carefully evaluating the project specifications and our team's technical strengths.

1.4.1.1 Java and Java Swing

Our team has decided to select Java as our chosen programming language together with its graphical user interface (GUI) library called Java Swing, especially for the development of the Santorini board game.

One of the main reasons for choosing Java is that all team members have experience coding in the language itself throughout our past assignments and personal projects. We are familiar with object-oriented programming (OOP) in Java which consists of several concepts such as polymorphism, inheritance, encapsulation, and abstraction. Besides that, we learned how to write efficient and maintainable code through the application of the SOLID principles, hence contributing to the enhancement of the software design and its robust system. We believe that all these concepts, skills, and other relevant knowledge are essential for the Santorini game development, considering the fact that the game is highly strategic with its comprehensive rules and complex mechanics. The game could also possibly have several extensions such as unique god powers, customizable board features, and the accommodation of multiple players. As such, the aforementioned concepts and principles enable us to make our game design modular and flexible without affecting the overall game logic and its core functionalities. We can also use suitable or appropriate design patterns in Java to organise our code neatly by dividing it into several components with focused responsibilities.

In addition, after doing some research and having group discussions, we agreed that we could fully utilise Java Swing for graphics rendering and animation in our project given its extensive set of components and layout managers (Prepbytes, 2023). The lightweight components including buttons, menus, and labels can be customised and they are easy to use (Prepbytes, 2023). Meanwhile, a variety of layout managers are provided to allow flexible arrangement of components on a GUI and build diverse, visually appealing user interfaces(Prepbytes, 2023). Not only that, Java Swing produces a more intuitive GUI through its robust event handling model to handle user interactions and inputs such as mouse clicks and keyboard presses to control the game elements, menus, and characters (Prepbytes, 2023). Furthermore, the Swing toolkit consists of a "pluggable look and feel" feature that allows us to customise the appearance and behaviour of the components in the GUI (Prepbytes, 2023). This provides a consistent user interface that is easy to navigate across numerous platforms.

As all team members are familiar with the Java programming language and its OOP concepts, it is not very likely that any one of us would require support from the teaching team. Regarding learning how to use the GUI, Java Swing for our project, we will do the necessary research and watch online tutorials to help us develop the essential components

needed for an interactive, scalable Santorini game application with an attractive user interface.

1.4.1.2 Python and Tkinter

We also considered another alternative which is Python along with its GUI called Tkinter. The main reason we chose Java over Python is because we have less experience coding with Python especially when it comes to game development. Although Python does support OOP, we have limited exposure to designing complex class hierarchies and applying object-oriented principles in our program. In comparison to Java, we are not very familiar with the OOP practices in Python such as abstract classes and method overriding as well as its syntax. This could cause us to work less efficiently and slow down our progress when coding the game for our project. Besides that, it may be harder for us to debug our code if multiple errors were encountered due to our lack of knowledge and experience in understanding the game logic and OOP patterns implemented in Python. Regarding its GUI, Tkinter, it has a few limitations as compared to Java Swing. Its outdated default widgets could make the GUI look less visually appealing and modern (Takle, 2025). Styling could appear to be inconsistent across the platforms (Takle, 2025). Tkinter offers limited support in customising the widgets or designing complex layouts due to its lack of advanced layout managers, which causes flickering and less responsiveness of user interfaces (Takle, 2025). Hence, learning how to overcome these limitations could consume time instead of focusing on implementing the essential game logic and design.

1.4.1.3 Final Selection

In conclusion, the team's final choice of programming language that will be used is **Java** and its GUI framework, namely **Java Swing**. All team members have prior exposure to Java and possess a solid understanding of the OOP concepts which are crucial to design a modular, maintainable architecture, given the game's complexity and extensibility. Java Swing also provides more flexibility and functionality than Tkinter due to its large set of components, advanced layout managers, and a reliable event-handling model which supports the development of a dynamic and engaging user interface. We considered using Python with Tkinter, however, the limited experience in Python's OOP and Tkinter's limited features for GUI development would delay the team's productivity in creating the game. Thus, Java is the most suitable choice for this project, as it aligns with our team's skill set and supports maintainable architecture, allowing us to build a stable, high-quality game while prioritizing user experience.

1.4.2 Technical Prototyping

Each team member's prototype is under their respective folders named after their own full name.

2.0 User Stories

This section presents the user stories for both the basic game rules and game extensions, ensuring a comprehensive understanding of core gameplay mechanics and additional features.

Each user story is carefully described and justified using the **INVEST** principle. The explanation of the INVEST principle is stated below.

- **I**ndependent: Each user story is self-sufficient and standalone with minimal dependencies.
- **N**egotiable: Each user story serves as a promise of a conversation, making them easy to change.
- **V**aluable: Every user story adds value to the player experience.
- **E**stimable: The user story should be clear with sufficient details for estimation.
- **S**mall: The user story should be small enough to fit within a sprint.
- **T**estable: Each user story has a clear definition of done through code reviews, functional tests, and acceptance tests.

Table 2.0.1 shows the justification of user stories.

Table 2.0.1 List of User Stories

<table>
<tr><th>No</th><th>User Story & Justification</th></tr>
<tr><td>1</td><td>As a player, I want to set up the game board with a cliff, ocean board, and 2 workers, so that I can start playing the game.

<table>
<tr><td>I</td><td>The setup of the game board can be implemented without relying on other features. It stands alone as a necessary step to begin the game.</td></tr>
<tr><td>N</td><td>This feature can be adjusted based on players’ feedback, additional requirements, and game balance considerations.</td></tr>
<tr><td>V</td><td>It ensures that players can quickly and easily start the game, enhancing the overall user experience.</td></tr>
<tr><td>E</td><td>This user story is clearly defined, as it involves setting up predefined board components, which makes it easy to estimate development time.</td></tr>
<tr><td>S</td><td>This setup feature is manageable and can be completed in a short time frame.</td></tr>
<tr><td>T</td><td>The correct setting of the elements mentioned can be verified through the interface.</td></tr>
</table></td></tr>
<tr><td>2</td><td>As a player, I want to select the colour of my worker character before the game starts, so that I can differentiate my workers and the opponent's workers during gameplay.

<table>
<tr><td>I</td><td>The colour selection of worker characters is a standalone feature that does not depend on other mechanics like movement or building. It can be developed separately.</td></tr>
</table></td></tr>
</table>

	N	A variety of colours can be selected based on the user's preference. This provides a room for negotiation as the appearance of the character can be changed.
	V	It enhances player experience by making it easier to distinguish between their own and their opponent's workers, improving clarity during gameplay.
	E	This feature is well-defined and clear, making it easy to estimate development time
	S	This user story is manageable within a sprint, as it involves adding a selection mechanic and some UI elements.
	T	This feature is testable because players can select the colour of the worker characters before the game starts and the selected colours are correctly displayed during gameplay.
3	As a player, I want to choose the initial locations for my workers, so that I can have a better start of the game.	
	I	Players selecting initial worker positions can function on their own without requiring modifications to other systems.
	N	There is room for negotiation on the ways of placing workers without fundamentally changing the core feature.
	V	It impacts gameplay by allowing players to choose an advantageous starting position, adding a layer of strategy rather than relying on random placement.
	E	This task is clearly defined, making it easy to estimate effort.
	S	This function is limited in scope, involving only the pre-game phase, and can be implemented within a single sprint.
	T	Success can be verified through functional tests, such as ensuring workers can be placed only in legal positions and confirming that the game correctly transitions from setup to gameplay.
4	As a worker, I want to move to an unoccupied adjacent space, so that I can reach better positions for future turns.	
	I	This moving feature is self-contained as it only involves the moving mechanism, which means it can be implemented separately.
	N	Discussions can be made to adjust or refine if needed.
	V	It provides strategic depth to the game, allowing players to position their workers effectively for future turns.
	E	The logic for moving a worker is straightforward, making it easy to estimate development time.
	S	This feature is manageable within a sprint, as it involves implementing movement rules and checking adjacency conditions.

	T	This moving feature can be verified through functional tests, such as ensuring workers can only move to unoccupied adjacent spaces and validating different movement scenarios.
5	As a worker, I want to climb one level up at a time, so that I can gradually build my way to the third level and secure a winning position.	
	I	This feature is self-contained as it only involves the climbing mechanism, which means it can be implemented separately.
	N	Discussions can be made to adjust or refine if needed.
	V	The ability to climb directly impacts the winning condition, making it essential for gameplay strategy.
	E	The logic for this user story is clear and quantifiable, requiring only movement validation based on current and target levels, making it easy to estimate development effort.
	S	It is a manageable feature within a sprint, as it involves checking the worker's movement constraints and updating their position accordingly
	T	The rule can be validated through functional and acceptance tests, ensuring that workers can only climb one level at a time and verifying edge cases.
6	As a worker, I want to move down any number of levels in a single turn, so that I can escape unfavorable positions and avoid being trapped.	
	I	The ability to move down levels is a standalone movement mechanic that can be implemented separately.
	N	The details of this rule can be adjusted based on game balance, such as restricting downward movement to a certain number of levels per turn or introducing specific conditions for moving downward.
	V	This feature prevents players from being trapped, allowing for more strategic movement. Without this rule, a worker on a high level with no available moves might become stuck, leading to frustrating gameplay.
	E	The feature can be clearly estimated in development time because it only requires modifying the movement rules to allow unrestricted downward movement while ensuring workers land on valid spaces.
	S	This feature is concise enough to be implemented in a sprint, involving movement logic updates and validation.
	T	This can be tested by creating scenarios where a worker attempts to move downward from different heights, ensuring that the movement follows game rules and does not cause rule-breaking behavior.
7	As a worker, I want to be restricted from making invalid moves, so that I can only make legal moves during the game.	

	I	This restriction on movement is a standalone rule that applies to all workers.
	N	The specifics of "invalid moves" can be adjusted.
	V	Preventing invalid moves ensures fair gameplay and prevents confusion.
	E	This feature involves defining movement constraints (e.g., adjacent movement, height restrictions, occupied spaces) and coding them into the game, making the effort required predictable.
	S	The implementation of this user story involves validating movement inputs against a set of predefined rules, making it manageable within a single development sprint.
	T	It can be tested by creating test cases where workers attempt illegal moves.
8	As a worker, I want to move one step before building a level structure in each turn so that I can follow the game's movement and building rules.	
	I	This user story applies to each worker individually and does not depend on other mechanics such as opponent actions or special abilities.
	N	The sequence of movement and building can be adjusted, for example, allowing certain abilities to modify this order in advanced game modes.
	V	This feature ensures that players follow the core gameplay rules, maintaining fairness and preventing unintended advantages by skipping movement or building twice in a turn.
	E	The effort required for this feature’s implementation is clear and measurable, involving turn structure validation to ensure movement occurs before building.
	S	It is specific and well-defined, making implementation manageable within a sprint by adding a turn-sequencing check in the game logic.
	T	It can be tested by attempting to build before moving, skipping movement, or making multiple builds in one turn to verify that the game enforces the correct sequence.
9	As a builder, I want to build a tower level structure on an adjacent space, so that I can gradually construct a path to victory.	
	I	This feature is self-contained as it only involves the building mechanism, which means it can be implemented separately.
	N	Discussions can be made to adjust or refine if needed.
	V	The ability to build directly influences the core strategy, allowing players to control movement and create opportunities to win.
	E	The logic for building a level structure is straightforward, making it easy to estimate development time.

	S	The feature is manageable within a sprint, as it involves implementing building rules and checking adjacency conditions.
	T	It can be verified through functional tests, such as ensuring workers can only build one level on the adjacent spaces and validating different building scenarios.
10	As a builder, I want to construct the tower level structure in the correct sequence, so that I can differentiate the actual level that I am standing on.	
	I	The construction of the correct tower level structure is self-contained, as constructing the correct level structure does not depend on other game mechanics beyond standard building rules.
	N	The way of visually representing a "correct level structure" can be discussed, in terms of shapes, colours and numbers.
	V	Ensures clarity in gameplay, preventing confusion about which level a worker is currently on, which is essential for legal movement and victory conditions.
	E	This user story is clear and measurable, involving proper level tracking and visual differentiation for each building stage.
	S	This feature is well-scoped, focusing on constructing levels correctly rather than broader mechanics like special powers or advanced structures.
	T	It can be verified by checking whether each level is placed correctly and whether workers can accurately determine their standing level.
11	As a builder, I want to be restricted from building in spaces occupied, so that I will not accidentally perform any illogical actions.	
	I	This user story does not depend on other mechanics beyond basic building rules, making it self-contained.
	N	The requirement is clear but it allows room for discussion about the exact details of the spaces occupied.
	V	It ensures fairness and clarity in gameplay by preventing illogical placements that could interfere with worker actions.
	E	The logic for restricting occupied spaces can be clearly implemented and tested.
	S	This feature is specific and manageable, focusing only on restricting building actions in occupied spaces.
	T	This feature can be verified by attempting to build in occupied spaces and confirming that the game prevents the action.
12	As a builder, I want to place a dome, so that I can ensure no players can stand on it.	
	I	The action of placing a dome does not rely on other mechanics beyond the core

	building rules.
N	This requirement can be discussed and modified as the user can negotiate the placement of the dome (e.g., on a tile, on top of the third level of the tower).
V	This feature prevents workers from standing on a certain tower after the dorm is placed, which adds a strategic layer to the game. This action can also restrict the opponent's movement by occupying the space.
E	This feature of building a dome is clear, making implementation effort easy to estimate.
S	It focuses on a single, well-defined action, making it manageable for development.
T	It can be tested by verifying that domes can be placed on any level and that workers cannot move onto domed spaces.

13 As a player, I want to move my worker onto the third level of a building when it is unoccupied, so that I can achieve victory according to the game rules.

I	The action of moving to the third level does not depend on other mechanics beyond movement rules.
N	The conditions for winning (e.g., special powers affecting movement) can be adjusted if needed.
V	This user story defines the core objective of the game, ensuring players have a clear path to victory.
E	It is straightforward (e.g., move to an unoccupied third level to win), making implementation effort easy to estimate.
S	This feature focuses on moving onto the third level, making it manageable for development.
T	This user story can be verified by checking if a worker reaches the third level and if the game correctly registers the victory condition.

14 As a player, I want the game to detect my loss when I have no possible moves or builds, so that I can prevent any unnecessary delays.

I	The detection of a loss due to no available actions does not depend on other features, which means this feature is a standalone feature.
N	If necessary, the conditions for losing, such as the inability to move or build, can be adjusted.
V	This feature prevents stalled gameplay and ensures a smooth experience by enforcing game-ending conditions.
E	The logic for checking legal moves and builds can be clearly defined and implemented.

	S	It focuses on a single, well-defined scenario for loss detection when no moves or builds are available.
	T	This feature can be verified by creating test cases where a player has no legal actions left and ensuring the game correctly registers the loss.
15	As a player, I want the game to enforce a turn-based system where I take my turn after my opponent, so that each player has an equal opportunity to make moves and the game remains fair.	
	I	The turn-based system applies to the game's turn structure and does not depend on other mechanics.
	N	The turn-based system can be modified if alternative game modes (e.g., multiple players, special abilities) are introduced.
	V	The feature of the turn-based system ensures fairness by preventing one player from playing multiple turns in a row.
	E	The implementation of this user story is straightforward, requiring the implementation of the logic of changing the player's turn.
	S	This feature is specific and focuses solely on maintaining the order of the player's turn.
	T	It can be verified by running scenarios where players attempt to take consecutive turns without switching, ensuring that the game correctly enforces turn-taking.
16	As a player, I want to restart the game after a player loses, so that I can start a new round.	
	I	Restarting the game is a standalone feature that does not rely on other mechanics beyond detecting a losing condition.
	N	The way of having the restart behavior can be adjusted. For example, the player can negotiate to restart the game automatically, allow a rematch option, or return to the main menu.
	V	This feature can eliminate downtime between rounds, keeping the gameplay smooth and engaging.
	E	The feature's implementation involves resetting the game board, player stats, and score tracking, which is clearly defined.
	S	This user story focuses only on restarting the entire game, making it manageable for development.
	T	This feature can be verified by checking whether the game resets correctly when the restart option is selected. For example, when the game is restarted, all elements should return to their initial state.
17	As an Artemis, I want to move two steps in a single turn, so that I can strategically	

	reposition myself by utilising my unique ability to move further.

I	This ability is a unique movement action exclusive to Artemis and does not affect other game mechanics.
N	The movement rules can be fine-tuned (e.g., limiting movement direction or allowing flexibility).
V	This provides a unique playstyle for Artemis, enhancing the strategic depth and playing experience.
E	This feature's implementation involves modifying movement rules for this specific character, making it clear to estimate.
S	The scope is limited to movement mechanics for Artemis, making it a manageable feature.
T	It can be verified by creating test cases to ensure Artemis correctly follows the two-step movement rule without violating other movement restrictions.

18	As a Demeter, I want to build twice on different spaces in a single turn, so that I can perform my unique ability to accelerate the tower construction.

I	This ability is a unique building action exclusive to Demeter and does not affect other game mechanics.
N	The restriction of not building in the same space can be adjusted if needed.
V	This user story adds strategic depth, allowing Demeter to manipulate the board more effectively.
E	The implementation of this feature requires modifying the build logic for Demeter, which is straightforward to estimate.
S	The scope is limited to an additional build action, making it manageable within a sprint.
T	It is testable as test cases can be created to verify whether the Demeter correctly builds twice in separate spaces within a single turn.

19	As a player, I want to receive a clear notification when I win or lose the game, so that I know that the game has ended.

I	Receiving notification while winning or losing a game operates separately from other game mechanics and does not depend on external elements.
N	The format of the notification can be adjusted based on user experience needs.
V	This user story helps players recognise when the game has ended, preventing confusion and unnecessary delays.
E	This feature is straightforward to implement with UI elements or alerts, making it

	easy to estimate development time.
S	This user story is small enough and is well-contained in adding a notification system, making it feasible within a sprint.
T	It can be verified by triggering win or loss conditions and checking whether the correct notification appears.

20 As a player, I want to be notified if I attempt to play in my opponent's turn, so that I can follow the turn-based rules and maintain fair gameplay.

I	Receiving notification during play turn does not depend on other mechanics beyond turn validation.
N	The notification method can be adjusted for a better user experience.
V	This user story prevents rule violations and ensures smooth, fair gameplay.
E	For this feature, clear logic and straightforward implementation make it easy to estimate development time.
S	A simple turn-checking mechanism can be implemented in a single sprint.
T	It can be verified by attempting to play out of turn and confirming that a notification appears.

21 As a player, I want to set the number of players before the game starts, so that I can accommodate different group sizes to have different playing experiences.

I	Changing the number of players is a self-contained feature that does not interfere with core mechanics.
N	This user story can be negotiated from the aspect of specifying the number of players in a round.
V	This user story adds flexibility, making the game more inclusive for different group sizes and enhancing the user experience while playing the game.
E	The complexity depends on handling multiple players and the corresponding UI updates, so it is manageable.
S	A single sprint can implement basic player count adjustments.
T	This feature can be verified by setting different player counts and checking if turns and rules function correctly.

22 As a player, I want to set the board size before the game starts, so that I can experience different levels of challenge and strategic depth.

I	Resizing the board does not affect other core mechanics.

	N	This feature is negotiable as the range of the board size can be discussed and adjusted.
	V	This user story allows for varied gameplay experiences, catering to different player preferences.
	E	This feature depends on implementing grid scaling and balancing mechanics, Therefore, it is small and feasible.
	S	A basic implementation of this user story (e.g., predefined board sizes) and further adjustment (e.g., different board sizes set) can be done within a sprint.
	T	It can be verified by setting different board sizes and ensuring movement, building, and winning conditions function correctly.
23	As a player, I want to have obstacles on tiles, so that I can increase the complexity of the game.	
	I	The addition of obstacles on tiles can be implemented on its own without relying on other mechanics in the game.
	N	The type, effect, and number of obstacles can be adjusted based on game balance. For example, a hole can be one of the choices of obstacle types.
	V	This user story adds value by increasing the complexity of the game, which could make it more engaging and challenging for players. It directly impacts the gameplay experience and can lead to more strategic thinking.
	E	The complexity of adding obstacles on tiles can be estimated as the difficulty of implementation can be determined by knowing the behaviors of the obstacles and interacting with other game elements.
	S	This feature is small and feasible, as it only requires implementing additional logic to define where the obstacle is placed and how the obstacle interacts with the player.
	T	This user story is testable as the effect of the obstacles can be verified through the interaction between them and the players.
24	As a player, I want to have different interactions with the tiles, so that I can create diverse gameplay scenarios and keep the game exciting.	
	I	Having different interactions with the tiles does not depend on other game mechanics or the implementation of other features.
	N	The details regarding the different interactions between the player and tiles can be refined and discussed. For example, the tile can restrict the player from moving and building on it.
	V	This story is valuable because it improves the player's experience by introducing a variety of tile interactions, which makes the game more engaging.
	E	The task is estimable because the concept of adding interactions to tiles is clear, and the general implementation of defining various tile behaviors and interactions is

		understood.
	S	This feature is achievable and can be completed within a single sprint.
	T	The story is testable because test cases can be written to validate that different interactions on tiles are working as expected.
25	As a player, I want to save and load my game to an external file, so that I can resume the game from where I left off.	
	I	This story is independent because it focuses on adding a specific feature that does not directly depend on other game mechanics. It can be developed and tested separately.
	N	The story is negotiable because it does not specify how the save/load feature should be implemented. The specific implementation details can be discussed and adjusted based on project needs.
	V	This feature provides clear value to the player, as it enhances the gaming experience by allowing users to save progress and continue their game at a later time.
	E	The task is estimable, as the concept of saving and loading a game from an external file is well-understood and has a clear implementation path.
	S	This feature is feasible and can be done within a sprint.
	T	This feature is testable. Test cases can be created to determine whether the game data is saved correctly and the player's progress is properly restored.

3.0 Domain Model

This section presents a detailed justification for the final version of the domain model that we have come up with, along with the iterations until we get a final version of the domain model.

In the rationale, we provide:

1. Justification for each chosen domain entity and their relationships
2. Choices we had to make while modelling the domain and its reason
3. Assumptions we have made, as well as any other part of our domain model that we feel warrants a justification as to why we modelled it that way.

; rationale for chosen domain model provided and discarded alternatives listed this means for iterations or rationale?

Rationale of the Domain Model

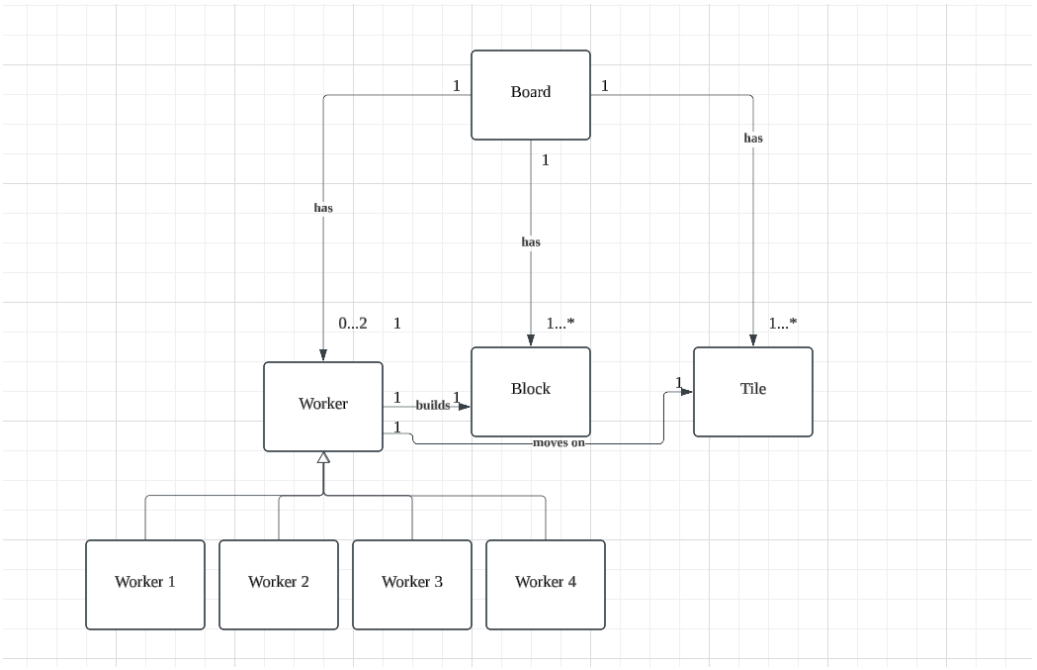
Domain entity	Justification	Relationships
Player	A participant in the game who selects the god power and mainly controls the actions of the two workers which include moving and building blocks during the player's turn.	● Composition with Worker: A player owns and controls two workers in the game, where the workers cannot exist independently without the player. If the player leaves the game, the workers are automatically removed as well. ● Aggregation with God Power: A player can choose a God Power before the game starts. The player does not own the god power, so the god power can exist independently without the player even if the player exits the game. It can be reused by other players in different game matches. ● Association with Reward: The player earns multiple rewards based on the achievements of their gameplay performance. So, the player is associated with the reward, making the reward an independent entity. If the player leaves the game, the reward still exists in the game system or game logs.
Worker	A game piece used by the player for interacting with the board. A worker executes game actions such as moving, building blocks, climbing up or down the blocks and activating the god power.	● Association with Obstacle: The worker is associated with the obstacle as the worker builds an obstacle during the game to hinder the opponent's movements. ● Association with Tile: The worker is associated with the tile as the current worker's position is indicated by the tile on the board. The worker moves by occupying one tile at a time. ● Association with Block: The worker is associated with the block through interactions such as moving up and down the blocks and building them.
God Power	An entity that assigns a specific role to the player and determines special abilities for them to modify the usual gameplay.	- aggregation with the player?? ● ● Generalization: Specific god powers such as Artemis and Demeter extend from the God Power entity which supports polymorphism. Artemis and Demeter can override the default behaviour in the parent class and implement their own unique logic. For instance, if the player

		selects Artemis as their god power, the player can move twice on the adjacent tiles. Meanwhile, if the player selects Demeter as their god power, the player can build blocks twice either on adjacent tiles or stack them.
Artemis (not sure if need or not)	A special god power that allows the workers to move twice during a single turn.	Mentioned in God Power
Demeter (not sure if need or not)	A special god power that allows the workers to build blocks twice during a single turn.	Mentioned in God Power
Board	It acts as the central game field which contains all tiles, facilitating worker movement, building construction and the gameplay mechanics.	• Association with Tile: The board is associated with the tiles as the board is composed of several tiles which defines a game grid for the gameplay to occur. The board dimensions can be configured by dynamically adjusting the number of tiles it contains. • Composition with tile?? Tile cannot exist independently without a board.., how to play the game if there is only one tile?
Tile	Each tile represents a unit of space on the board, where workers can move, construct buildings and build obstacles.	• Association with Worker: The tile is associated with the worker as it indicates the current worker's position on the board. The tile is occupied by the player one at a time. • Association with Block: The tile is associated with a number of blocks, as the building is constructed from level 1 to 4 on the tile. • Association with Obstacle: The tile is associated with the obstacle as the obstacle is built on the tile to block the opponents.
Block	Represents a building component which can be stacked on a tile to form a building structure	• Generalization: Through generalization, Building Level and Block become subclasses of the Block entity with shared attributes and override the abstract methods to implement their own behaviour, hence supporting polymorphism. For instance, building levels can be built and climbed up one at a time while a dome cannot be climbed

		upon and built on top of it by another worker.
Building Level	A level which can be constructed on a tile or stacked on another level	Mentioned in Block
Dome	A hemisphere that represents the final top level of the building structure	Mentioned in Block
Reward	Represents a collectible element or in-game achievement earned by a player based on their performance.	Mentioned in Player
Obstacle	A game component placed on a tile to restrict player movement for an enhanced strategic gameplay.	Mentioned in Worker and Tile
Game Controller (do we need this class, is it included in the problem space of the domain model)	It manages the entire game logic, including: ➢ Player setup ➢ Board initialization ➢ Tracking the player's turn ➢ Validate game rules ➢ Checking win or loss conditions ➢ Handles transitions between game states	• Association with Player: The Game Controller is associated with Player as it is responsible for creating a number of Player objects and assigning the workers to the players • Association with Board: The Game Controller is associated with Board as it interacts with the game space to monitor all the game actions together with the rules that occur throughout the game.

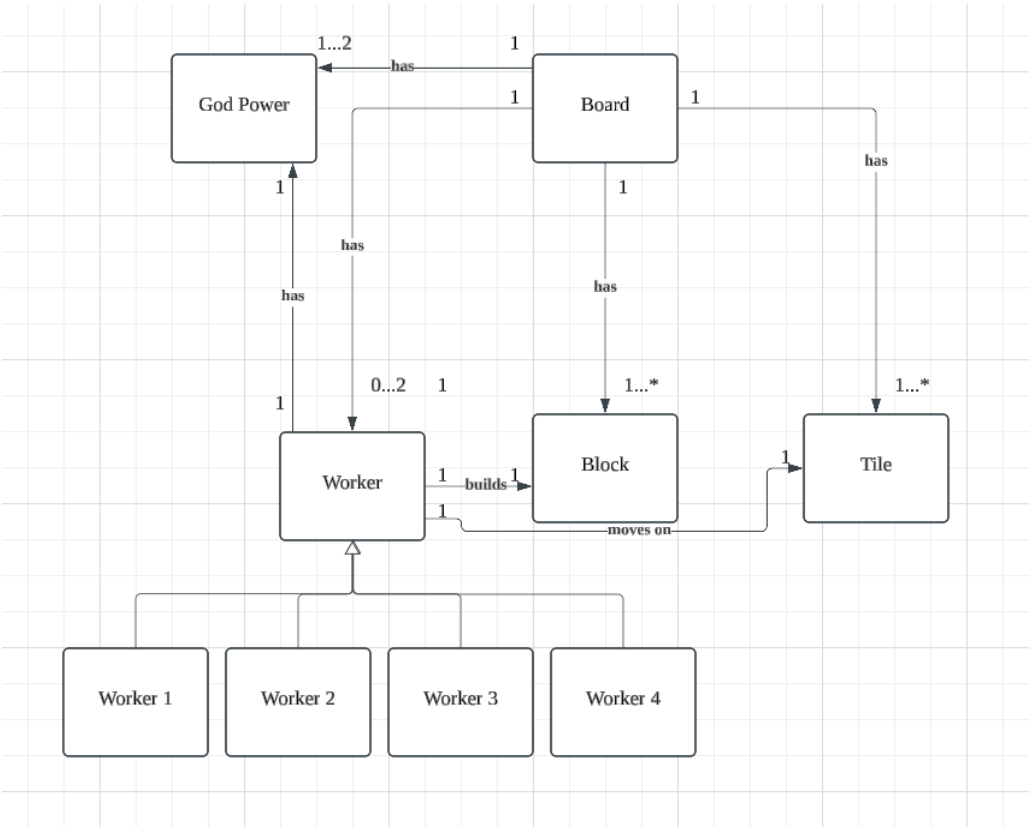
Iteration	Domain Model

1



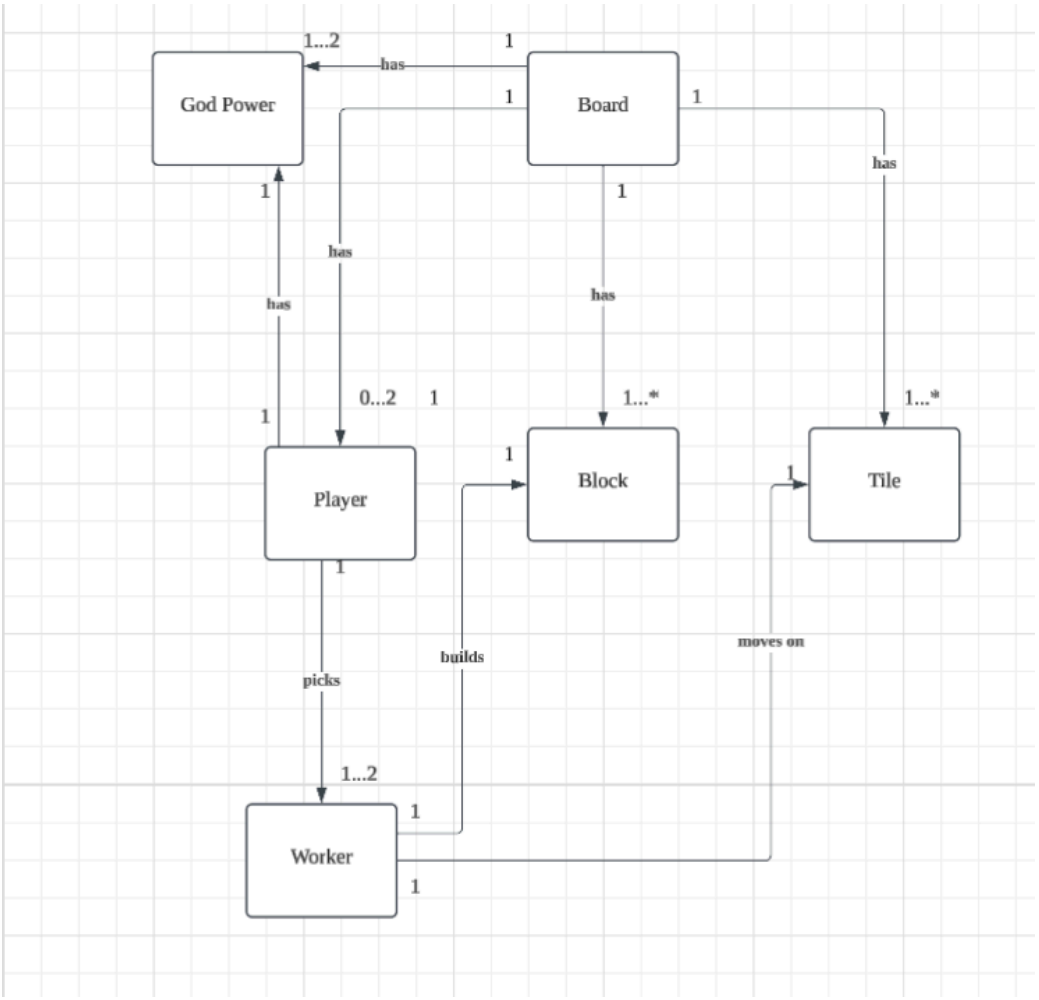
In this domain model, there are 4 entities which show the initial setup of Santorini. The Board entity is connected to Worker, Block, and Tile entities because they are placed on top of the Board. Worker entity has 4 sub entities, which are Worker 1, Worker 2, Worker 3 and Worker 4, as our initial thought is that we can use a Worker entity to inherit all the 4 workers to simplify our model, making the model more neat. Here, Worker is connected to Block and Tile so that worker walks on tiles and builds the block. Though there is an issue, we missed out the characters (Artemis and Demeter) and their God Power which are included in the basic Santorini gameplay.

2

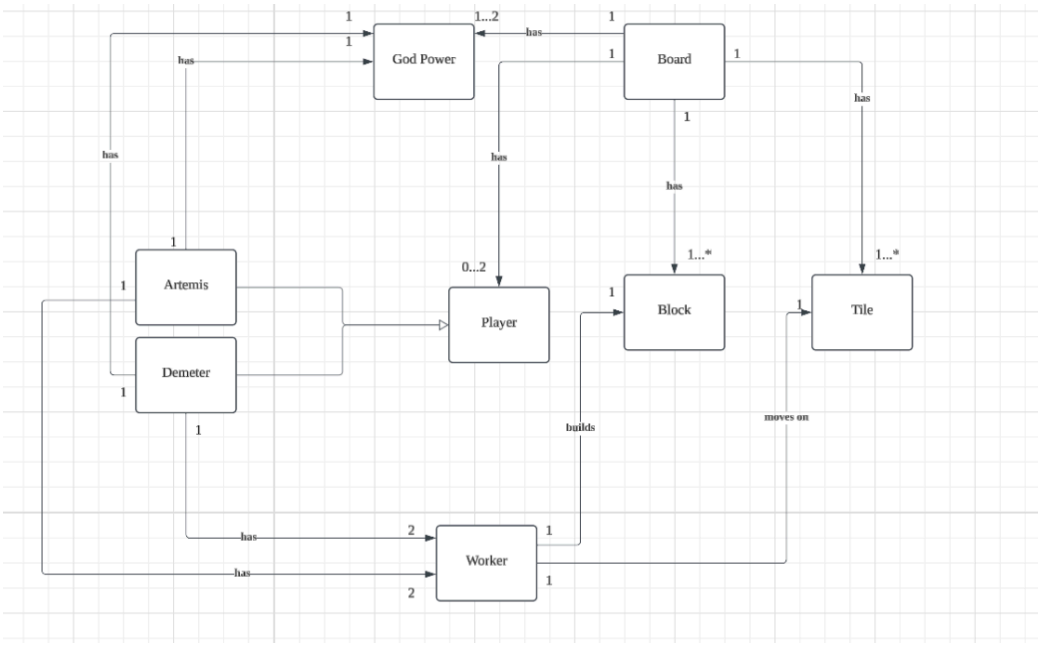


An entity God Power is added, with the Board connecting to it as power cards are placed on the board too. Besides, Worker is connected to God Power as worker is able to perform unique actions based on God Power. We discovered a problem here, as God Power is obtained only by characters, but not workers.

3



This model we added a Player entity to represent the players playing the game. It could also represent characters who have the God Power, as players can pick one character to play. Here, Worker can build the Block and move on the Tile.

4	 To further classify the existence of characters, we generalise Artemis and Demeter to the Player entity, showing that characters are inherited by the Player, meaning the player who chose which character is able to have the character's God Power when playing the game. Problem encountered here is that, instead of connecting Artemis and Demeter entities to Worker and God Power, we can connect by using Player since Player entity is the parent class of them.

3.0 Domain Model

This section presents a detailed justification for the final version of the domain model that we have come up with, along with the iterations until we get a final version of the domain model.

In the rationale, we provide:

1. Justification for each chosen domain entity and their relationships
2. Choices we had to make while modelling the domain and its reason
3. Assumptions we have made, as well as any other part of our domain model that we feel warrants a justification as to why we modelled it that way.

; rationale for chosen domain model provided and discarded alternatives listed this means for iterations or rationale?

Rationale of the Domain Model

Domain entity	Justification	Relationships

Player	A participant in the game who selects the god power and mainly controls the actions of the two workers which include moving and building blocks during the player's turn.	● Composition with Worker: A player owns and controls two workers in the game, where the workers cannot exist independently without the player. If the player leaves the game, the workers are automatically removed as well. ● Aggregation with God Power: A player can choose a God Power before the game starts. The player does not own the god power, so the god power can exist independently without the player even if the player exits the game. It can be reused by other players in different game matches. ● Association with Reward: The player earns multiple rewards based on the achievements of their gameplay performance. So, the player is associated with the reward, making the reward an independent entity. If the player leaves the game, the reward still exists in the game system or game logs.
Worker	A game piece used by the player for interacting with the board. A worker executes game actions such as moving, building blocks, climbing up or down the blocks and activating the god power.	● Association with Obstacle: The worker is associated with the obstacle as the worker builds an obstacle during the game to hinder the opponent's movements. ● Association with Tile: The worker is associated with the tile as the current worker's position is indicated by the tile on the board. The worker moves by occupying one tile at a time. ● Association with Block: The worker is associated with the block through interactions such as moving up and down the blocks and building them.
God Power	An entity that assigns a specific role to the player and determines special abilities for them to modify the usual gameplay.	- aggregation with the player?? ● ● Generalization: Specific god powers such as Artemis and Demeter extend from the God Power entity which supports polymorphism. Artemis and Demeter can override the default behaviour in the parent class and implement their own unique logic. For instance, if the player selects Artemis as their god power, the player can move twice on the adjacent

		tiles. Meanwhile, if the player selects Demeter as their god power, the player can build blocks twice either on adjacent tiles or stack them.
Artemis (not sure if need or not)	A special god power that allows the workers to move twice during a single turn.	Mentioned in God Power
Demeter (not sure if need or not)	A special god power that allows the workers to build blocks twice during a single turn.	Mentioned in God Power
Board	It acts as the central game field which contains all tiles, facilitating worker movement, building construction and the gameplay mechanics.	• Association with Tile: The board is associated with the tiles as the board is composed of several tiles which defines a game grid for the gameplay to occur. The board dimensions can be configured by dynamically adjusting the number of tiles it contains. • Composition with tile?? Tile cannot exist independently without a board.., how to play the game if there is only one tile?
Tile	Each tile represents a unit of space on the board, where workers can move, construct buildings and build obstacles.	• Association with Worker: The tile is associated with the worker as it indicates the current worker's position on the board. The tile is occupied by the player one at a time. • Association with Block: The tile is associated with a number of blocks, as the building is constructed from level 1 to 4 on the tile. • Association with Obstacle: The tile is associated with the obstacle as the obstacle is built on the tile to block the opponents.
Block	Represents a building component which can be stacked on a tile to form a building structure	• Generalization: Through generalization, Building Level and Block become subclasses of the Block entity with shared attributes and override the abstract methods to implement their own behaviour, hence supporting polymorphism. For instance, building levels can be built and climbed up one at a time while a dome cannot be climbed upon and built on top of it by another worker.

Building Level	A level which can be constructed on a tile or stacked on another level	Mentioned in Block
Dome	A hemisphere that represents the final top level of the building structure	Mentioned in Block
Reward	Represents a collectible element or in-game achievement earned by a player based on their performance.	Mentioned in Player
Obstacle	A game component placed on a tile to restrict player movement for an enhanced strategic gameplay.	Mentioned in Worker and Tile
Game Controller (do we need this class, is it included in the problem space of the domain model)	It manages the entire game logic, including: ➢ Player setup ➢ Board initialization ➢ Tracking the player's turn ➢ Validate game rules ➢ Checking win or loss conditions ➢ Handles transitions between game states	• Association with Player: The Game Controller is associated with Player as it is responsible for creating a number of Player objects and assigning the workers to the players • Association with Board: The Game Controller is associated with Board as it interacts with the game space to monitor all the game actions together with the rules that occur throughout the game.

4.0 Basic UI Design

4.1 Main Menu

This section shows the basic UI design for the main menu of the game.

The main menu of the game.

4.2 Interaction of game

This section shows the basic UI of the interaction of the game.

Player 1 choose the player that they want according the colour.

Player 2 choose the player that they want according the colour.

The player was prompted to choose whether they wanted to add a god card for each player.

If the player did not want a god card.

If player choose to have a god card.